Requirement on academic honesty (COMP2090SEF):

1. Each group must make the following declaration on the cover page of the report

Hong Kong Metropolitan University

COMP2090SEF
Data Structures, Algorithms And Problem Solving

Course Project Report

Group

Submission date:

We declare that:
(i) all members of the group have read and checked that all parts of the project (including proposal, code programs, and reports), irrespective of whether they are contributed by individual member or all members as a group, here submitted is original except for source material explicitly acknowledged;
(ii) the project, in parts or in whole, has not been submitted for more than one purpose without declaration;
(iii) we are aware of the University's policy and regulations on honesty in academic work and understand the possible consequence when breaching such policy and regulations;
(iv) we confirm that we have declared in the report about the usage of AI and other generative models, including but not limited to ChatGPT, LLaMA, Gemini, Mistral, and Stable Diffusion, and complied with the instructions provided by HKMU; and
(v) we are aware that all members of the group should be held responsible and liable to disciplinary actions, irrespective of whether he/she has signed the declaration and whether he/she has contributed, directly or indirectly, to the problematic contents.

We confirm that we have read through and understood the project requirements. We understand that failure to comply with the project requirements will result in score deduction.

NAME	SID
Laiyutao	13996440
Longenze	14018180
Zhangwancen	14002864

2. The group must acknowledge ALL external resources or code from others referenced. We will only mark the parts that are original, not the parts from others.

3. All members are liable to the plagiarism incidence, even if a free-rider.

Core Functions

This project is a food donation management system named FoodBridge, built with Flask and Flask-SQLAlchemy. Its core functions are:

User Management: Supports registration, login, and role-based access control (donor/receiver). Donors can publish surplus food listings, while receivers can claim available food.

Food Listing Management: Donors can create food listings with details such as title, description, location, and expiry time. The system automatically calculates expiry time based on user input.

Claim & Pickup Workflow: Receivers can claim available food listings; after successful claim, the system notifies the donor via email/SMS. Both donors and receivers can confirm the completion of food pickup.

Dashboard: Role-specific dashboards display relevant data (donors see their published listings, receivers see their claimed food).

Data Persistence: Uses SQLite database to store user information, food listings, and workflow statuses.

Techniques Summary

The project leverages the following key techniques:

Flask Web Framework: A lightweight Python web framework for routing, request handling, template rendering, and session management.

Flask-SQLAlchemy: An ORM extension for Flask that abstracts database operations, enabling model definition and CRUD (Create, Read, Update, and Delete) operations without writing raw SQL.

Flask-Login: Manages user authentication, session maintenance, and access control for protected routes.

Object-Oriented Programming (OOP): Encapsulates business logic into models and service classes, using design patterns such as Factory, Singleton, and Strategy.

Notification Service Layer: Separates notification logic from route handlers, supporting multiple notification channels (email, SMS) via polymorphic design.

Usage of OOP Concepts

1. Encapsulation

Encapsulation is used to hide internal implementation details and expose only necessary interfaces:

Model Encapsulation: The User class encapsulates user data and password security. It uses private attri *bute*(*e.g.*$_{u}$*sername*)*andpubl* ic methods like set_password() and check_password() (verifies hashed passwords), preventing direct access to sensitive data.

Business Logic Encapsulation: The FoodListing class encapsulates state management logic. Methods like claim receiver id and complete_pickup() handle state transitions (e.g.,

checking availability before claim, validating permissions before pickup), keeping route code clean and focused on request handling.

2. Inheritance & Polymorphism

Inheritance: The FoodListing base class is extended by subclasses like PerishableFood and NonPerishableFood, which inherit common attributes (title, location, expiry time) while implementing type-specific behaviors.

Polymorphism: The FoodFactory creates different FoodListing subclasses based on the food_type parameter. Each subclass implements the display_info() method differently, allowing the system to show type-specific information without modifying route code.

Strategy Pattern Polymorphism: Notification classes (EmailNotifier, SMSNotifier) implement a common notify() interface. The NotificationManager calls notify_all(), which invokes the appropriate notify() method for each notifier, demonstrating polymorphic behavior.

3. Factory Pattern

The FoodFactory class centralizes the creation of FoodListing objects. The create_food() method takes a food_type parameter and returns the corresponding subclass instance, decoupling object creation from route logic and making it easy to add new food types without changing existing code.

4. Singleton & Service Layer

The NotificationManager is implemented as a singleton to ensure a single instance manages all notification services. It uses lazy initialization to load notifiers only when needed, reducing resource overhead. This service layer encapsulates all notification logic, allowing routes to simply call notify_all() without knowing the underlying implementation details of email/SMS sending.

Declaration of External Resources

To facilitate robust development, security, and maintainability, this project relies on a selection of open-source Python packages and web frameworks. All dependencies are formally documented and managed within the application's requirements.txt environment file. The external resources utilized include:

- **Flask (v3.0.0):** Served as the primary web application framework. It was utilized to handle backend HTTP routing, request/response parsing, and the overall model-view-controller (MVC) architectural flow.

- **Flask-SQLAlchemy (v3.1.1):** Implemented as the Object-Relational Mapper (ORM). This package enabled the program to interact with the SQLite database using Python object-oriented constructs. It was critical for facilitating our Single - Table Inheritance design pattern.

- **Flask-Login (v0.6.3):** Used for managing persistent user sessions, authentication, and access control. It provided security decorators to ensure the

correct separation of privileges between the "Donor" and "Receiver" dashboard views.

- **Werkzeug (v3.0.1):** A comprehensive WSGI web application utility library. Specific cryptographic functions from this package (generate_password_hashand check_password_hash) were integrated into our User class encapsulation methods to securely encrypt user credentials within the database.
- **Jinja2 Engine:** Integrated directly via Flask, this templating engine was used to dynamically inject back-end Python logic—such as object properties, database query results, and live expiration timestamps—into the front-end HTML structures.
- **Bootstrap / System UI Utilities:** External CSS design paradigms were referenced to rapidly build a responsive and modernized frontend interface (e.g., utilizing utility classes for tables, buttons, and alert flashes) without relying purely on out-of-the-box browser default styling.

Self-reflection (Potential Weaknesses & Future Improvements)

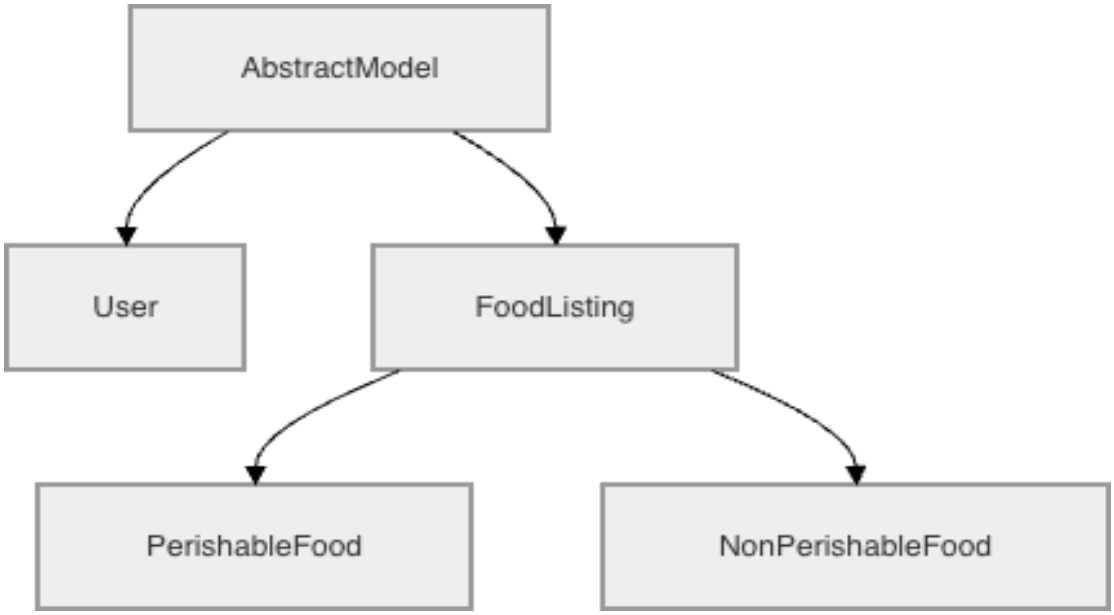
Potential Weaknesses

- Limited Scalability: The app uses SQLite, which cannot support high-concurrency production environments or large-scale data growth effectively.
- Insufficient Input Validation: There are no comprehensive checks for user inputs (e.g., food expiry dates, location formats), which may cause invalid data or security risks like XSS/SQL injection.
- Poor User Experience: The app lacks pagination for food listings, frontend styling, and confirmation prompts for critical actions (e.g., claiming food), which reduces usability.

Future Improvements

- Technical Optimization: Replace SQLite with PostgreSQL to improve scalability; add input validation and security protections against common web vulnerabilities.
- Function Enhancement: Add food search/filter (by location/type), auto-archiving for expired listings, and user-specific notification preferences.
- UX Upgrade: Implement pagination for food listings, add frontend styling for better visual appeal, and include confirmation modals for key operations (e.g., claiming/completing food pickup).
- OOP Refinement: Refactor core logic with dependency injection for better maintainability; add an AdminUser subclass to expand role-based permission management.

Appendix A **UML Class Diagram**

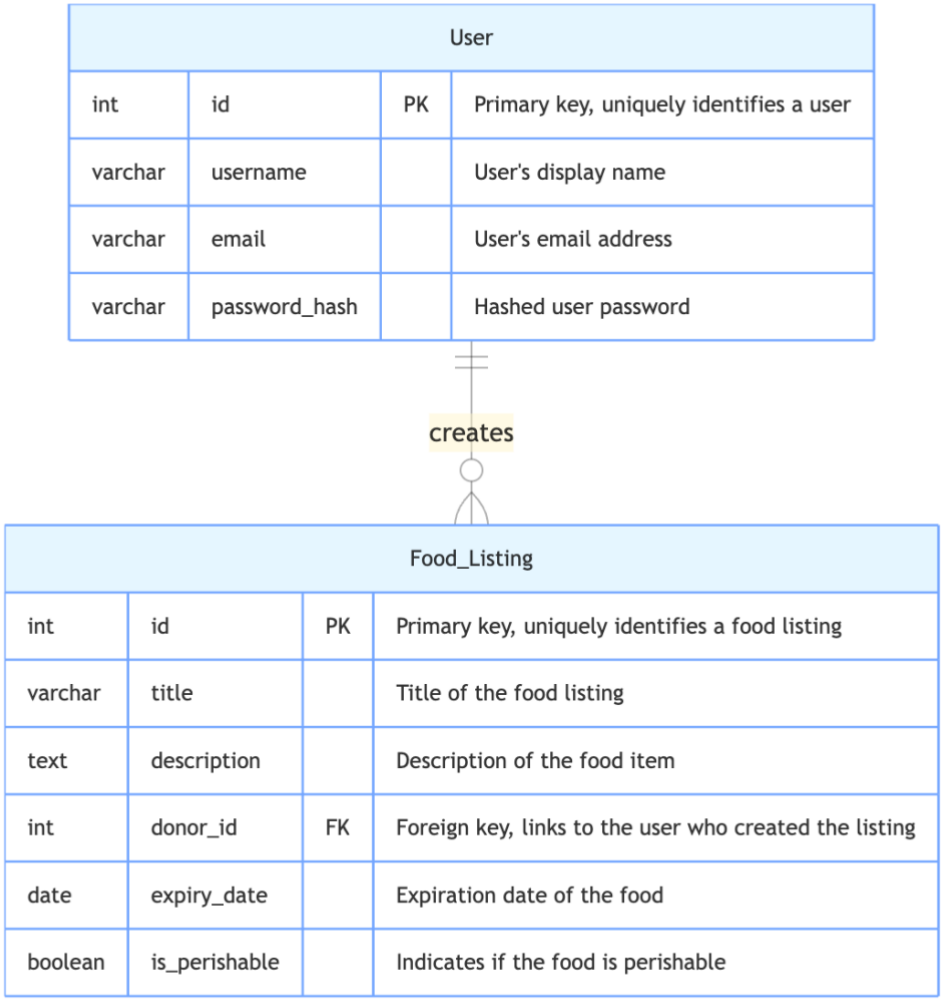


Appendix B **Application of Core OOP Concepts**

OOP Concept	Application in Codebase	Explanation / Benefit
Abstraction	AbstractModel class (models.py)	Hides internal ORM implementation details and enforces a standard API template across the codebase. Compels all inheriting entities to define a customized display_info() representation.
Encapsulation	User class properties (models.py)	Protects sensitive data variables. The __password_hash is highly restricted. Properties like @username.setter bind business logic directly to data assignment (e.g., rejecting usernames under 3 characters).
Inheritance	Single-Table Inheritance (models.py)	PerishableFood and NonPerishableFood share the common attributes of an overarching FoodListing template, drastically reducing code duplication while sharing the same underlying SQLite table structure.

Polymorphism	display_info() and Magic Methods	Subclasses react differently to the same method call. A PerishableFood prints a completely different UI format than a NonPerishableFood. Magic methods like __lt__ allow lists of custom food objects to be seamlessly sorted by the Python engine.

Appendix C



Appendix D

Weakness Type	Corresponding Improvement	User Experience Impact
Technical Limitation	Migrate to PostgreSQL; add security validation	Ensures stable, secure app performance
Core Function & UX Gap	Add search/filter, pagination, styling & modals	Directly improves usability & convenience
Maintainability & Security	Refactor OOP logic; add role-based permissions	Sustains long-term UX reliability